Representation and semantics

Arithmetic combines finite-width representations with operation rules. An adder processes bit patterns. Signed interpretation, instruction semantics, and status controls determine what happens when a mathematical result is outside the representable range.

The chapter develops integer ALUs, iterative multiply/divide hardware, and IEEE 754 floating point. Every model needs operand width, result width, rounding, exceptions, and timing assumptions.

Interpret `0xffffffff + 1` both ways before discussing its circuit.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The arithmetic problem

A fixed 32-bit word cannot represent every integer, fraction, or real number. Overflow, underflow, rounding, and special values are part of a representation contract.

Hardware trades latency, throughput, area, and precise exceptions. A multiplier can generate more partial products in parallel. A floating-point adder retains extra bits for rounding, which requires additional datapath and pipeline state.

Define the required result before choosing a gate diagram.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.1 The basic model

Modular integer instructions compute in the ring modulo 2^n. Trapping and saturating instructions impose different policies. Floating point maps an exact arithmetic result into a finite representable set F.

$$y = \operatorname{round}(x \circ z).$$

The rounding function, exceptional conditions, and special values are part of the operation. Correct rounding does not imply an infinitely precise mathematical result.

Integer division by zero depends on the ISA or language. IEEE floating-point `0/0` is invalid and ordinarily produces a quiet NaN when no trap is enabled.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.2 Two's-complement addition and subtraction

$$x = -b_{n-1}2^{n-1} + \sum_{i=0}^{n-2} b_i 2^i.$$

The low n result bits of an adder are the same for signed and unsigned addition. Whether they represent the mathematical signed sum depends on overflow.

$$a - b = a + \overline{b} + 1 \pmod{2^n}.$$

Complementing the second input and setting carry-in to one lets subtraction share an adder.

A correct modular bit pattern can still be an unrepresentable signed mathematical result.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Deriving signed overflow

Let the operand signs be s_a, s_b and the result sign be s_r. Equal operand signs with a changed result sign indicate overflow.

$$ov = \overline{s_a \oplus s_b} \wedge (s_a \oplus s_r) = c_{\text{into MSB}} \oplus c_{\text{out of MSB}}.$$

Unsigned overflow depends only on carry out. `0x7fffffff+1` overflows signed addition. `0xffffffff+1` overflows unsigned addition while signed −1 plus 1 is valid.

Never replace both tests with an unqualified "overflow equals carry."

Bit-level addition and subtraction

For an eight-bit adder, let c_7 enter the sign position and c_8 leave it. Signed overflow is $ov = c_7 \oplus c_8$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Deriving signed overflow (continued)

Operation	Adder inputs	Result	c_7	c_8	Unsigned carry	Signed overflow
`0x7f+0x01`	`01111111+00000001`	`10000000`	1	0	0	1
`0x80+0xff`	`10000000+11111111`	`01111111`	0	1	1	1
`0x80-0x01`	`10000000+11111110+1`	`01111111`	0	1	1	1
`0xff+0x01`	`11111111+00000001`	`00000000`	1	1	1	0

The last row has a carry without signed overflow. `slt` and `sltu` also interpret identical bits differently. Subtraction uses $a + \bar{b} + 1$, with overflow $(s_a \oplus s_b) \wedge (s_a \oplus s_r)$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Adder timing and exceptions

A ripple-carry adder propagates carry along the width. Lookahead and parallel-prefix designs shorten this dependency using more gates and wiring.

An overflowing `add` must not commit an ordinary destination result in place of its exception. `addu` retains the modular result without the signed-overflow trap.

Clock design uses the worst valid path and setup/hold constraints, not average operand behavior. Result generation and exception detection can have distinct paths.

Draw both the carry path and the overflow-detection path.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Saturating multimedia arithmetic

Saturation clamps an out-of-range result to an endpoint. Signed eight-bit $100 + 100$ saturates to 127 instead of wrapping to a negative value.

This avoids wraparound but loses reversibility. It cannot replace modular arithmetic arbitrarily in checksums or cryptographic algorithms.

Packed SIMD arithmetic operates on several short lanes. Carries do not cross lane boundaries. Lane width and saturation policy belong to the instruction semantics.

Audio clipping and raw byte addition require different numerical contracts.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.3 Multiplication hardware

An exact unsigned $n \times n$ product can require $2n$ bits. Two 32-bit inputs therefore need up to a 64-bit result.

$$AB = \sum_{k=0}^{n-1} b_k (A \ll k).$$

Shift-and-add accumulates a shifted multiplicand for each set multiplier bit. Shifting handles a power of two. Generating arbitrary partial products also requires bitwise gating.

Truncating an input or partial product before the exact computation changes the problem.

Sequential multiplication and correctness

For unsigned n-bit M and Q, inspect Q0, conditionally compute $C : A \leftarrow A + M$, and shift the full (C, A, Q) right. C enters A's high bit. For M=13 and Q=11:

Round	Q0	Add result C:A	A after shift	Q after shift
0			`0000`	`1011`
1	1	`0:1101`	`0110`	`1101`
2	1	`1:0011`	`1001`	`1110`
3	0	`0:1001`	`0100`	`1111`
4	1	`1:0001`	`1000`	`1111`

The final product is `10001111` =143. Combined states R0..R4 are 11,109,158,79,143.

Fix the initial multiplier $q < 2^n$, and define $R_k = 2^n A_k + Q_k$ after round k. The exact invariant is

$$R_k = \left\lfloor \frac{q}{2^k} \right\rfloor + M(q \bmod 2^k)2^{n-k}, \qquad 0 \le k \le n.$$

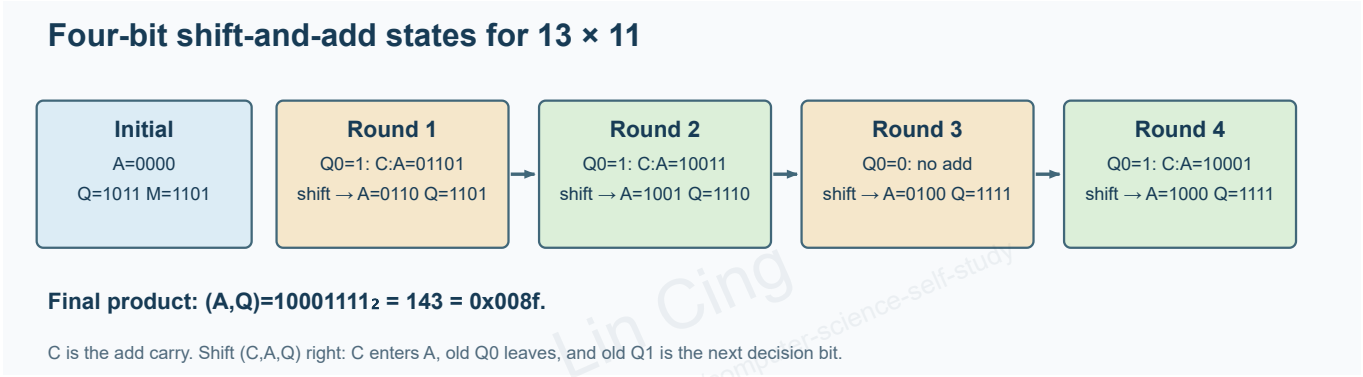
Sequential multiplication and correctness (continued)

At $k = 0$, $A_0 = 0, Q_0 = q$, so it holds. When $k < n$, the second term is even. The current low bit is therefore original multiplier bit q_k. The conditional addition produces full combined value $R_k + q_k M 2^n$, including the carry. Shifting gives

$$R_{k+1} = \left\lfloor \frac{R_k + q_k M 2^n}{2} \right\rfloor = \left\lfloor \frac{q}{2^{k+1}} \right\rfloor + M(q \bmod 2^{k+1}) 2^{n-k-1}.$$

The equality uses $q \bmod 2^{k+1} = (q \bmod 2^k) + q_k 2^k$, completing induction. At $k = n$, $q < 2^n$ makes the first term zero, so $R_n = Mq$. A contains the high half and Q the low half. Carry C enters A during the shift and is not lost. The processed old Q0 leaves; old Q1 becomes the next decision bit.

Sequential multiplication and correctness (continued)



$$T_{\text{serial}} = t_{\text{setup}} + n t_{\text{iter}} + t_{\text{finalize}}.$$

Thirty-two iterations are 32 cycles only if each add and shift fits one cycle. Separate-cycle operations require a different cost. A parallel tree reduces dependent additions while adding wires, compressors, and power. Pipelining reduces initiation interval without necessarily reducing one operation's latency.

Signed multiplication

One method multiplies unsigned magnitudes and applies the product sign. The most negative input needs care because its positive magnitude does not fit in the same signed width, although it fits in the corresponding unsigned width.

An exact signed product fits in $2n$ signed bits. Booth recoding instead derives signed partial products directly from two's-complement input. Radix-4 recoding reduces their count while adding selection and sign-extension complexity.

Test $(-2^{n-1})^2$ rather than assuming that ordinary signed absolute value is always representable.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Faster multiplication

Parallel multipliers generate partial products together and compress them with carry-save structures or trees. Carry-save representation avoids a full-width carry propagation at every compression level.

A final carry-propagate adder produces the conventional result. Reduced depth costs area, routing, and switching power.

A pipelined multiplier can accept independent inputs more frequently than its result latency. A scoreboard or hazard mechanism still protects consumers of unfinished products.

One new multiply per cycle and three-cycle result latency are compatible claims.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Traditional MIPS multiply instructions

`mult` and `multu` write a 64-bit product into HI and LO. `mfhi` and `mflo` move the halves into general-purpose registers.

Signed and unsigned hardware can share components, but upper product bits have different mathematical interpretations. Traditional `mult` and `multu` do not trap merely because the result exceeds 32 bits.

Taking only LO gives a low modular product. Exactness requires checking the full result against the intended result range.

`multu` interprets input bits as unsigned values. It is not an assertion about source-language types.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.4 Division hardware

Unsigned division seeks Q, R satisfying

$$D = Qd + R, \qquad 0 \le R < d, \quad d > 0.$$

Signed division additionally specifies quotient truncation and remainder sign. Truncation toward zero generally makes the nonzero remainder share the dividend's sign.

Division by zero and `INT_MIN / -1` need explicit policies. Traditional MIPS32 `div` and `divu` do not automatically raise arithmetic traps for these cases. Their HI/LO results must not be relied on. Software must check them. Other architectures and languages can specify different behavior.

Check zero divisor, signs, rounding direction, and the minimum-integer boundary separately.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Restoring division

Each unsigned iteration shifts the combined remainder/quotient state left and subtracts the divisor. Keep a nonnegative trial and emit quotient bit 1; otherwise restore the shifted remainder and emit bit 0.

Fix n=4, M=5, Q=1101, and A=0. The five-bit A preserves the trial sign. Each completed round has $0 \leq A < M$.

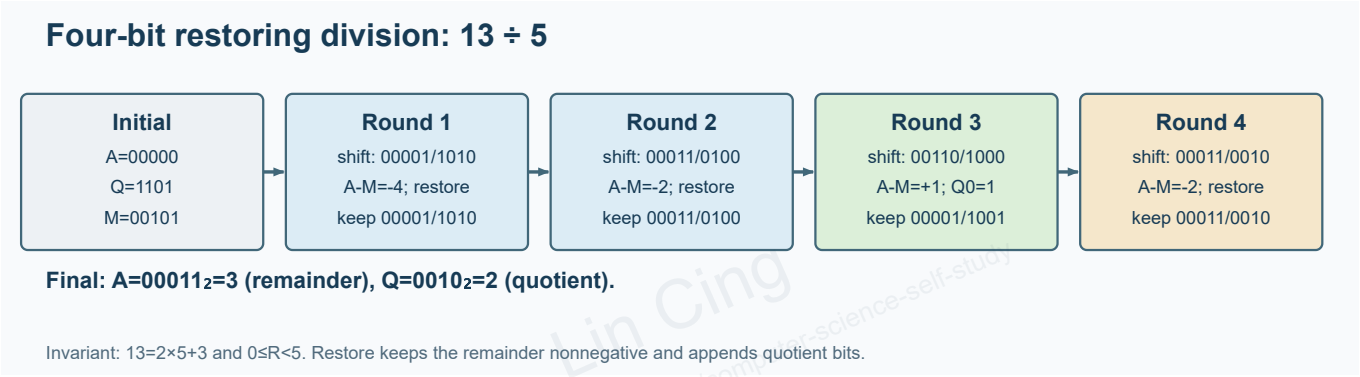
Restoring-division invariant

Let P_k be the dividend prefix already consumed, Q_k the generated quotient, and A_k the remainder. Maintain

$$P_k = Q_k M + A_k, \qquad 0 \leq A_k < M.$$

For the next dividend bit d, candidate remainder is $2A_k + d < 2M$. If it is at least M, append quotient bit 1 and subtract M; otherwise append 0 and keep it. Both branches preserve the equality for prefix $P_{k+1} = 2P_k + d$, and retain the remainder bound. After every dividend bit, $D = QM + A$ with $0 \leq A < M$.

Restoring division (continued)



For D=13,M=5, the final A=3 and Q=2 satisfy $13 = 2(5) + 3$. This combined-register variant uses n rounds. A different hardware arrangement may use n+1 rounds; do not mix their iteration counts, although their final pair must agree.

Signed and faster division

Magnitude division followed by sign correction is one signed implementation. It must preserve the specified quotient and remainder convention.

SRT division selects quotient digits from an estimate and residual recurrence. Radix-4 can generate two quotient bits per iteration, but digit selection, normalization, and correction add work.

Leading-zero detection and early termination can reduce work for selected operands. These mechanisms cannot change architectural results.

Half as many iterations need not mean twice the speed. Per-iteration time and fixed overhead also matter.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Divide instructions

Traditional MIPS `div` and `divu` put the quotient in LO and the remainder in HI. `mflo` and `mfhi` retrieve them.

Latency and initiation interval differ. A compiler can replace division by a constant with reciprocal multiplication or shifts only when the full input range and rounding behavior remain equivalent.

For signed −7, arithmetic right shift by one yields −4, whereas division by 2 with truncation toward zero yields −3. Adding a bias of 1 before shifting corrects this case: (−7+1) arithmetic-shift-right 1 = −3. Generalizing the transformation requires the matching divisor, signed width, bias, and overflow conditions.

Verify negative and minimum-value inputs before accepting a divide replacement.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.5 IEEE 754 representation

$$value = (-1)^s \, significand \, 2^e.$$

Binary32 has 1 sign bit, 8 exponent bits, and 23 fraction bits. Normal values use exponent fields 1 through 254, bias 127, and an implicit leading 1.

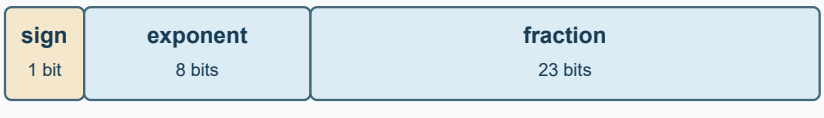
Binary64 has fields of 1, 11, and 52 bits, with bias 1023. Exponent range and significand precision solve different representation problems.

"Floating point" alone does not specify a format.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Floating-point fields

IEEE 754 binary32 fields and encoding of -12.5



0xC1480000

sign=1; exponent=10000010₂

-12.5 = -1.1001₂ × 2³

value = (-1)^s × 1.fraction × 2^(exponent-127)

Zero exponent uses subnormal rules. All-one exponent denotes infinity or NaN; hidden 1 is not universal.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

For a normal number,

$$value = (-1)^s (1.F)_2 \, 2^{E-bias}.$$

Precision is 24 bits for binary32 and 53 for binary64, including the leading bit. With zero exponent field, a subnormal has leading bit zero. Binary32 subnormals use exponent −126 and provide gradual underflow toward zero.

Floating-point fields (continued)

The hidden leading 1 does not apply to subnormal numbers.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Special values and flags

An all-ones exponent with zero fraction denotes signed infinity. A nonzero fraction denotes NaN. Quiet and signaling NaNs have distinct invalid-operation behavior and payload rules.

IEEE exception classes include overflow, underflow, inexact, invalid, and division by zero. A flag and a language exception are not the same mechanism. Trap controls determine whether ordinary nontrapping continuation occurs.

$+\infty + (-\infty)$ is invalid and ordinarily returns quiet NaN. $1/(+\infty) = +0$ under the usual IEEE rules.

Separate the numerical result, status flag, and enabled trap.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Floating-point addition: alignment precedes rounding

For finite normal inputs, compare exponents and shift the smaller significand to a common scale. Opposite signs require magnitude subtraction; the larger magnitude determines the sign. Information shifted beyond retained precision must survive until rounding.

Normalize the sum or difference, round to the target precision, and renormalize if rounding carries out. Special values, zeros, overflow, and subnormal results need their own paths.

The following reduced model has **p=4 significant bits including the leading 1, RNE, and an unbounded exponent range**. It exposes every intermediate bit; it is not binary32.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

An alignment trace where sticky changes the answer

Let a=1 and b=9/128=0.0703125. Both are exact at p=4. Aligning $b=1.001_2\times2^{-4}$ gives $0.0001001_2\times2^0$.

p=4 including the leading 1; RNE; unbounded exponent range

×2⁰	Retain 4 bits	G	R	Tail bits
a = 1	1.000	0	0	00
b = 9/128	0.000	1	0	01
a + b	**1.000**	**1**	**0**	**01**

$b = 1.001_2 \times 2^{-4} = 0.0001001_2 \times 2^0$

S = OR(0,1) = 1 → GRS = 101 → 1.001_2 = 1.125

Replacing the tail by 00 would create a false tie and return 1.000_2.

The exact sum is 137/128. Its adjacent candidates are 1 and 9/8, with midpoint 17/16=136/128. RNE selects 9/8 with error 7/128. Returning 1 after discarding the tail would give error 9/128, exceeding half an ULP=1/16.

Cancellation: find the leading 1 again

At p=4, compute exact 0.75−0.6875. After subtraction, shift left three positions.

p=4, RNE: 0.75 − 0.6875; both inputs are exact

a	**1.100 000**	× 2^{-1}
− b	**1.011 000**	× 2^{-1}
Subtract	**0.001 000**	× 2^{-1}
Shift left 3	**1.000 000**	× 2^{-4}

Each left shift decreases the exponent by one; value is unchanged.

GRS=000; the result 1/16=0.0625 is exact, so inexact=0.

The difference is exact. Cancellation can amplify existing input error without creating rounding error. At the minimum exponent, use subnormals.

Derive nearest-even from G, R, and S

Work on the result's magnitude. L is the least retained bit, G the next bit, and R the following bit. S is the OR of every remaining bit, summarizing whether the tail is nonzero.

In retained-ULP units, G has weight 1/2, R has weight 1/4, and the tail is below 1/4. G=0 is below midpoint. G=1,R=S=0 is exactly halfway. G=1 with R or S nonzero is above halfway. At a tie, choose the candidate with even retained encoding. Thus

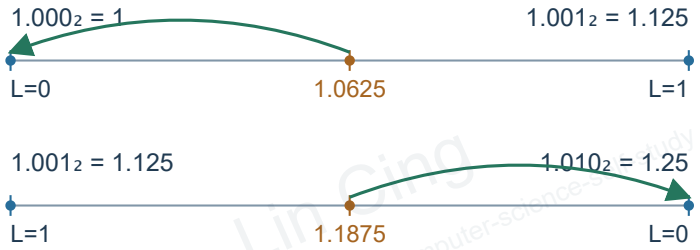
$$\text{inc} = G \wedge (R \vee S \vee L).$$

If inc=1, increment the retained **magnitude** by one ULP, then restore the sign. For a negative result this can make the number more negative; it is not always rounding toward +∞. Any nonzero discarded part implies inexact.

Two midpoints, two parity decisions

Even refers to the significand encoding: the intervals below select opposite endpoints.

p=4, RNE; each number line shows one adjacent interval



inc = G ∧ (R ∨ S ∨ L)

Round the magnitude, then restore the sign: −1.1875 → −1.25.

At p=4, 1.1111_2 has L=1,GRS=100. Increment: $1.111_2+0.001_2=10.000_2=1.000_2\times2^1$. Renormalization increases the exponent.

Two binary32 rounding checkpoints

Binary32 has p=24 and spacing 2^{-23} on [1,2). For $1+2^{-24}$, L=0 and GRS=100 select 1 (0x3f800000). For $1+3\cdot2^{-25}$, GRS=110 selects $1+2^{-23}$ (0x3f800001). Both operations set inexact.

Binary32 round-to-nearest-even and five FP exceptions

Tie: $1+2^{-24}$	Above half: $1+3\cdot2^{-25}$	Underflow tie: 2^{-150}	Overflow: max+max
GRS=100, retained LSB=0	GRS=110	midpoint: 0 and 2^{-149}	exact result > max finite
candidate: 1.0	round up: $1+2^{-23}$	even→+0	rounded→+∞
inexact	inexact	underflow + inexact	overflow + inexact

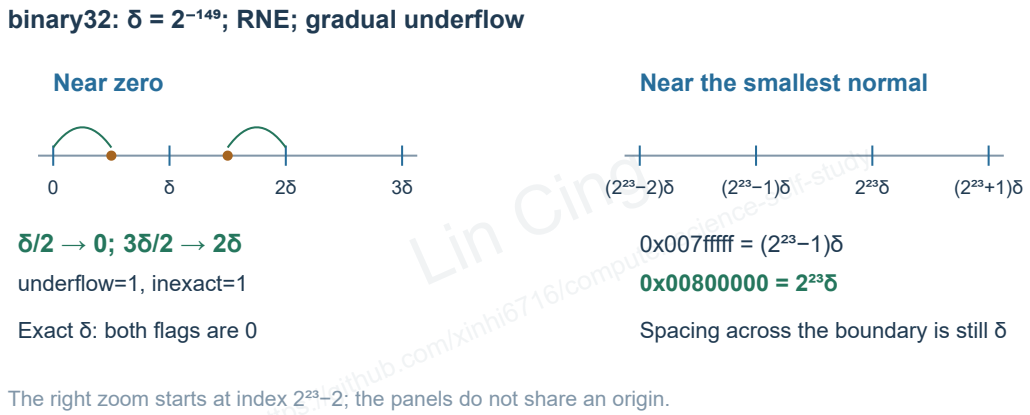
Invalid: 0×∞, ∞−∞, 0/0 → quiet NaN; 1/±0 → signed infinity.

G/R/S = guard/round/sticky. Ties select retained LSB zero; NaN follows payload and comparison rules.

Here 2^{-24} is half the adjacent-value spacing. FMA rounds exact ab+c once, whereas separate multiply and add have two rounding points. A later worked example compares their output bits.

Subnormal representation and underflow flags are different questions

Put $\delta = 2^{-149}$. Positive binary32 subnormals are $F\delta$ for fraction integers $1 \le F \le 2^{23}-1$. The smallest normal is $2^{23}\delta = 2^{-126}$. Spacing remains δ across this boundary.



The right zoom starts at index $2^{23}-2$; the panels do not share an origin.

Subnormal does not imply an underflow flag. Exact $2^{-126}/2$ gives 0x00400000 without error; the two inexact midpoint cases set both flags. The next page specifies the environment.

Check exactness at the subnormal boundary

Fix RNE, gradual underflow, disabled traps, initially clear flags, and tininess after rounding. Every input is exact in binary32. UF and NX denote underflow and inexact.

Operation	Exact value	RNE output bits	UF	NX
$2^{-126} / 2$	2^{-127}	0x00400000	0	0
$\delta / 2$	$\delta/2$	0x00000000	1	1
$(3\delta) / 2$	$3\delta/2$	0x00000002	1	1
Smallest normal − largest subnormal	δ	0x00000001	0	0

At the midpoints, even fraction integers win: choose 0 between 0 and 1, and 2 between 1 and 2. Flags record conditions, not error magnitudes. Flush-to-zero or treating subnormal inputs as zero changes this model.

See Oracle Numerical Computation Guide § 2.3 for representation and gradual underflow.

Special values: report the result and the exception

Maximum finite plus itself returns +∞ under RNE, setting overflow and inexact. The invalid operations $0 \times \infty$, $\infty - \infty$, and $0/0$ return quiet NaN when traps are disabled.

Finite nonzero divided by signed zero returns infinity with the XOR of the input signs, setting divide-by-zero. Infinity divided by +0 remains infinity without that flag. See the Oracle Numerical Computation Guide exception table.

NaN comparison depends on the predicate: ordinary equality is false even for a NaN compared with itself; inequality is true. Distinguish ordered and unordered predicates. +0 and −0 compare equal, but reciprocal and signbit distinguish them.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Floating-point multiplication

For normal inputs, multiply $(1.F_a)(1.F_b)$, add unbiased exponents, and XOR signs. The exact significand product lies in $[1, 4)$. Normalize and round it to target precision.

Subnormal inputs may require normalization before multiplication. A tiny result can enter the gradual-underflow path. Special combinations are handled first. In particular, $0 \times \infty$ is invalid.

Multiplication avoids exponent alignment but needs a wide significand product and final rounding.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

MIPS floating-point instructions

The book's model uses coprocessor-1 registers. `lwc1` and `swc1` move words. `add.s` and `add.d` select single and double formats.

Comparisons set condition information examined by instructions such as `bc1t` and `bc1f`, with the traditional control-transfer delay slot. NaNs require ordered and unordered comparison semantics.

Floating-point and general-purpose register spaces differ. The compiler must respect register format and saved-area requirements.

These encodings and register rules are not interchangeable with ARM or RISC-V.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Accuracy and rounding modes

Nearest-even is a common default. Directed modes round toward positive infinity, negative infinity, or zero. Directed rounding can support interval bounds.

An ULP is the spacing between adjacent representable values in the relevant binade. Small absolute error does not imply small relative error. Spacing grows with exponent.

Significand width controls precision. Exponent width controls range. Adding exponent bits does not add significand accuracy.

Compare binary32 and binary64 spacing at the same value rather than comparing their largest values alone.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

NaNs, signed zero, and comparisons

NaN is unequal to every value under ordinary equality, including itself. Use `isnan` or an unordered comparison to detect it.

IEEE equality treats positive and negative zero as equal, but reciprocals, sign-copy operations, and serialization can expose their different sign bits.

$\infty - \infty$, $0 \times \infty$, and ∞/∞ are invalid. Signed integer ordering of raw bits cannot implement ordinary floating-point ordering across negative values and NaNs.

A comparison algorithm must specify its treatment of NaNs and signed zeros.

3.6 Parallelism and nonassociativity

Integer addition remains associative modulo a fixed power of two. Floating-point rounding can instead make

$$(a + b) + c \neq a + (b + c).$$

A changed reduction tree changes rounding points. Different answers can therefore be a numerical reproducibility issue without any data race. Bitwise reproducibility requires a fixed order and matching formats, rounding, intermediate precision, and FMA contraction, or an accumulator with an explicit order-independent guarantee. Compensation and higher precision improve error but do not alone guarantee order independence.

For standard Kahan summation with each operation rounded to binary32 RNE, `[2^25,1,-2^25]` gives 0, while `[2^25,-2^25,1]` gives 1. In the first order, adding 1 leaves the sum at 2^{25} and compensation at -1. Then $-2^{25} + 1$ is a midpoint and rounds back to -2^{25}, producing zero. The second order cancels exactly before adding 1. Compensation has not removed dependence on order or environment.

3.6 Parallelism and nonassociativity (continued)

FMA versus two rounding points

At $x = 2^{24}$, the binary32 ULP is 2. `x+1` is the midpoint between x and x+2, and the even candidate is x:

$$(x + 1) - x = 0, \qquad (x - x) + 1 = 1.$$

These expressions agree over exact reals but differ under per-operation rounding. Let $a = b = 1 + 2^{-23}$, bits `0x3f800001`, and $c = -(1 + 2^{-22})$, bits `0xbf800002`.

$$ab = (1 + 2^{-23})^2 = 1 + 2^{-22} + 2^{-46}.$$

Separate multiply and add first round the product to $1 + 2^{-22}$, then cancel with c to +0. A fused operation computes exact $ab + c = 2^{-46}$ before its sole rounding, yielding `0x28800000`. It changes the intermediate rounding, not real algebra. The old MIPS core does not guarantee FMA; an extension must specify format and rounding mode.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.7 The x86 floating-point case

The book compares x87 extended-precision stack registers with SSE2 XMM scalar and packed operations. Keeping an intermediate wider than its memory format can differ from rounding every operation to that format.

ABI rules, compiler flags, spills, and precision controls affect the rounding points. SIMD operations also require correct lane and alignment semantics.

Do not silently substitute modern AVX behavior for a historical x87 example. State the actual arithmetic environment.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.8 Arithmetic fallacies

Approximation has exact rules. Applications can depend on rounding and exception flags.

Fast-math transformations can permit reassociation or assumptions about NaNs and signed zero. They can change reproducibility and numerical behavior.

There is no universal fixed ratio between divide and multiply costs. Operand dependence, pipelines, specialization, and instruction mix all matter.

Pair each performance claim with a specified operation and timing model.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study/

3.9 Arithmetic synthesis

Integer width, overflow, and modular behavior must be specified together. Multiply/divide iteration count does not by itself determine latency or throughput.

A floating-point unit implements representation, arithmetic, and rounding with exceptions. Addition emphasizes alignment and lost-bit information. Multiplication emphasizes the significand product and normalization.

Verification needs normal values, zeros, subnormals, infinities, NaNs, overflow, underflow, and halfway cases.

Ordinary exact sums alone do not exercise the rounding contract.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study/

Numerical history

Earlier machines used incompatible exponent widths, biases, and representations. IEEE 754 standardized formats, rounding, and special values to make numerical behavior more predictable.

Hardware evolved through iterative arithmetic, recoding, compression trees, pipelined floating point, and SIMD. This did not eliminate numerical analysis. Ill-conditioning, large dynamic range, and reproducible parallelism remain algorithmic concerns.

Separate format, arithmetic hardware, rounding, and numerical algorithm.

Lin Cing
https://github.com/xinhi6716/computer-science-self-st

End-of-chapter check

Determine signed and unsigned overflow, trace small multiply and divide states, decode binary32 special cases, and justify guard/round/sticky decisions. Give a counterexample to floating-point associativity.

Use exercises and solutions separately. Show representation and state transitions, not only the final decimal number.

A bit-level trace makes both the arithmetic policy and its boundary cases inspectable.

Lin Cing
https://github.com/xinhi6716/computer-science-self-st

References

Patterson and Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, Revised Fourth Edition, 3.1–3.10, printed pp. 222–283 (PDF pp. 249–310). See the unit guide for edition and unavailable-material boundaries.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

David A. Patterson, John L. Hennessy. Computer Organization and Design: The Hardware/Software Interface, Revised Fourth Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study